How I Deployed a Secure Web Application on AWS using Ansible and Load Balancing

---

Read This First (Important)

This guide is written for a **complete beginner**.

Name: *Odo Kingsley Uchenna.*

Role: *Cloud Engineer.*

Brand: *RealKingHubs.*

We will:

- Go step by step
- Explain *what to click*, *what to type*, and *why*
- Build real Cloud Engineer skills

Do **not** skip steps.
Do **not** rush.

STEP 0 — What We Are Building (Very Simple Explanation)

Before touching AWS, understand the goal.

You will build:

- **1 Bastion Host** (public server)
- **2 Web Servers** (private servers)
- **1 Application Load Balancer (ALB)**
- **NGINX installed automatically using Ansible**
- **A website accessible ONLY through the Load Balancer**

You will **NOT**:

- ❌ SSH directly into web servers from your laptop
- ❌ Access the website using EC2 public IPs

This is how real companies design secure systems.

STEP 1 — Create an AWS Key Pair (SSH Key)

This key allows you to securely log in to servers.

What to do

1. Log in to AWS Console
2. Go to **EC2**
3. Left menu → **Key Pairs**
4. Click **Create key pair**

Settings

- **Name**: cloud-project-key
- **Key pair type**: RSA
- **Private key format**: .pem

Click **Create key pair**

The file downloads automatically.

VERY IMPORTANT (Do this on your computer)

Move the key into your SSH folder and secure it:

```
mkdir -p ~/.ssh
mv ~/Downloads/cloud-project-key.pem ~/.ssh/
chmod 400 ~/.ssh/cloud-project-key.pem
```

Why?
SSH will refuse to use the key if permissions are too open.

STEP 2 — Create a VPC (Networking Foundation)

We need **public and private networks**.

What to do

1. Go to **VPC**
2. Click **Create VPC**
3. Select **VPC and more**

Configuration

- **Name**: cloud-project-vpc
- **IPv4 CIDR**: 10.0.0.0/16
- **Availability Zones**: 2
- **Public subnets**: 2
- **Private subnets**: 2
- **NAT Gateway**: 1 per AZ
- **VPC endpoints**: None

Click **Create VPC**

Wait until creation finishes.

Why?
Private servers need internet access (for updates) but must not have public IPs.

STEP 3 — Create Security Groups (Firewall Rules)

We will create **3 security groups**.

STEP 3.1 — Bastion Security Group

1. Go to **EC2** → **Security Groups**
2. Click **Create security group**

Settings

- **Name**: bastion-sg
- **VPC**: cloud-project-vpc

Inbound rules

Type	Port	Source
SSH	22	Your Public IP

Outbound: Allow all

Click **Create**

Only YOU can SSH into the bastion.

STEP 3.2 — Web Server Security Group

Create another security group.

Settings

- **Name**: web-sg
- **VPC**: cloud-project-vpc

Inbound rules

Type	Port	Source
SSH	22	bastion-sg

Outbound: Allow all

We will add HTTP later.

STEP 3.3 — Load Balancer Security Group

Create another security group.

Settings

- **Name**: alb-sg
- **VPC**: cloud-project-vpc

Inbound rules

Type	Port	Source
HTTP	80	0.0.0.0/0

Outbound: Allow all

STEP 4 — Launch EC2 Instances (Ubuntu, t3.micro)

4.1 Bastion Host

1. EC2 → **Launch instance**

Configuration

- Name: bastion-host
- AMI: Ubuntu Server 22.04 LTS
- Instance type: t3.micro
- Key pair: cloud-project-key

Network

- VPC: cloud-project-vpc
- Subnet: Public subnet 1
- Auto-assign public IP: Enable
- Security group: bastion-sg

Click **Launch**

4.2 Web Server 1

- Name: web-server-1
- AMI: Ubuntu 22.04
- Instance type: t3.micro
- Key pair: same key

Network

- Subnet: Private subnet 1

- Auto-assign public IP: Disable
- Security group: web-sg

Launch instance

4.3 Web Server 2

Repeat the same steps:

- Name: web-server-2
- Private subnet 2
- No public IP
- web-sg

Launch instance

STEP 5 — Test SSH Access (VERY IMPORTANT)

SSH into Bastion

```
ssh -i ~/.ssh/cloud-project-key.pem ubuntu@<BASTION_PUBLIC_IP>
```

If it works → GOOD

From Bastion → Web Server

Inside bastion:

```
ssh ubuntu@<WEB_SERVER_PRIVATE_IP>
```

If this works → networking is perfect.

STEP 6 — Install and Prepare Ansible (Beginner Level)

Important rule: Ansible is installed on **your local machine**, not on the bastion host.

6.1 Check if Ansible is already installed

Open a terminal on your local machine and run:

ansible --version

- If you see a version number → Ansible is already installed
- If you get `command not found` → continue to Step 6.2

6.2 Install Ansible (Ubuntu – beginner safe)

Run these commands **one by one**:

sudo apt update
sudo apt install -y ansible

Verify installation:

ansible --version

Why Ansible?
Instead of logging into each server and installing software manually, Ansible lets us configure **multiple servers automatically** from one place.

Note: Sometimes the ansible version that comes in is not the latest version which might lead you to hours of debugging.
To Fix it: Upgrade Ansible

Run these commands on your Vagrant box to install the latest version via the Official Ansible PPA

sudo apt update
sudo apt install software-properties-common -y
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install ansible -y

6.3 Create a Project Folder

We keep everything organized.

```
mkdir aws-ansible-project
cd aws-ansible-project
```

Create required files:

```
touch inventory.ini deploy-nginx.yml
```

STEP 7 — Configure SSH Access for Ansible (ProxyJump)

Ansible uses SSH. Since our web servers are private, we tell SSH:

First go to the **bastion host**
Then jump to the **private web servers**

7.1 Edit SSH config file

```
nano ~/.ssh/config
```

Paste the following (**replace IPs with your real values**):

```
Host bastion
  HostName <BASTION_PUBLIC_IP>
  User ubuntu
  IdentityFile ~/.ssh/cloud-project-key.pem

Host web1
  HostName <WEB_SERVER_1_PRIVATE_IP>
  User ubuntu
  IdentityFile ~/.ssh/cloud-project-key.pem
  ProxyJump bastion

Host web2
  HostName <WEB_SERVER_2_PRIVATE_IP>
  User ubuntu
  IdentityFile ~/.ssh/cloud-project-key.pem
  ProxyJump bastion
```

Save and exit.

7.2 Test SSH connections (VERY IMPORTANT)

Run:

```
ssh web1
```

Exit and test:

```
ssh web2
```

If both work → networking + SSH are perfect
If not → stop and fix before continuing

Why this matters: If SSH fails, Ansible will fail.

STEP 8 — Create Ansible Inventory

Open inventory file:

```
nano inventory.ini
```

Paste:

```
[web]
web1
web2
```

Save and exit.

This tells Ansible: *“These are my web servers.”*

STEP 9 — Write the Ansible Playbook (NGINX + HTML)

Open playbook file:

nano deploy-nginx.yml

Paste:

```
---
- name: Configure web servers
  hosts: web
  become: yes

  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes

    - name: Install NGINX
      apt:
        name: nginx
        state: present

    - name: Start and enable NGINX
      service:
        name: nginx
        state: started
        enabled: yes

    - name: Deploy custom HTML page
      copy:
        dest: /var/www/html/index.html
        content: |
          <!DOCTYPE html>
          <html>
          <head>
            <title>Cloud Project</title>
          </head>
          <body>
            <h1>Welcome To My Secure Web Application</h1>
            <p><strong>Name:</strong> Your-Name</p>
            <p><strong>Hostname:</strong> {{ ansible_hostname }}</p>
            <p><strong>Private IP:</strong> {{ ansible_default_ipv4.address }}</p>
            <p>About Me: Beginner Cloud Engineer learning AWS, Linux and DevOps.</p>
          </body>
          </html>
```

What this playbook does (simple):

- Updates Ubuntu
- Installs NGINX
- Starts NGINX automatically
- Creates a unique webpage per server

STEP 10 — Run the Ansible Playbook

From the project folder:

```
ansible-playbook -i inventory.ini deploy-nginx.yml
```

Expected result:

- No errors
- `ok` and `changed` messages

At this point:

- NGINX is installed automatically
- Website is live on both private servers
- No manual installation was done

STEP 11 — Test From Bastion (Optional Verification)

SSH into bastion:

```
ssh bastion
```

From bastion:

```
curl http://<WEB_SERVER_1_PRIVATE_IP>
curl http://<WEB_SERVER_2_PRIVATE_IP>
```

You should see **different hostnames and IPs**.

STEP 12 — Create the Application Load Balancer (ALB)

Now we create the **only public entry point** to the application.

Users will access the website **only through the Load Balancer**.

12.1 Create a Target Group

Go to **EC2 → Target Groups**

Click **Create target group**

Settings:

- Target type: **Instances**
- Target group name: **web-target-group**
- Protocol: **HTTP**
- Port: **80**
- VPC: **cloud-project-vpc**
- Health check path: **/**

Click **Next**

Register targets:

- Select **web-server-1**
- Select **web-server-2**
- Click **Include as pending**

Click **Create target group**

Why this matters:
The target group tells the Load Balancer **where to send traffic**.

12.2 Update Web Server Security Group

Now we allow HTTP traffic **only from the Load Balancer**.

Go to **EC2 → Security Groups**

Open **web-sg**

Inbound rules → **Edit** → **Add rule**:

- Type: **HTTP**
- Port: **80**
- Source: **alb-sg**

Save rules.

Web servers are still private.

STEP 13 — Create the Application Load Balancer

Go to **EC2** → **Load Balancers**

Click **Create load balancer**

Choose **Application Load Balancer**

Basic configuration

- Name: **web-alb**
- Scheme: **Internet-facing**
- IP address type: **IPv4**

Network mapping

- VPC: **cloud-project-vpc**
- Select **both public subnets**

Security groups

- Select **alb-sg**

Listeners

- Protocol: **HTTP**
- Port: **80**
- Forward to: **web-target-group**

Click **Create load balancer**

Wait until status shows **Active**.

STEP 14 — Verify Target Health

Go to **EC2** → **Target Groups**

Click **web-target-group**

Open **Targets** tab

You should see:

- web-server-1 → **Healthy**
- web-server-2 → **Healthy**

If unhealthy:

- Check security groups
- Ensure NGINX is running
- Health check path must be **/**

STEP 15 — Final Browser Test (VERY IMPORTANT)

Copy the **ALB DNS name**

It looks like:

web-alb-123456789.us-east-1.elb.amazonaws.com

Open browser and visit:

http://<ALB_DNS_NAME>

Refresh the page multiple times

You should see:

- Different hostnames
- Different private IPs

This proves:

- Load balancing works
- High availability is achieved
- Web servers remain private

STEP 18 — CLEAN UP (MANDATORY)

Delete in this order to avoid unnecessary charges from AWS:

1. Load Balancer
2. Target Group
3. EC2 Instances
4. NAT Gateway
5. VPC

Prevents AWS charges.

This is how I deployed a secure, highly available web application on AWS using Ubuntu, Ansible, a bastion host, private subnets, and an Application Load Balancer.

As a beginner, I often wished for a clear roadmap like this. I'm sharing these guidelines today to help others navigate the same path. This is a solid look into Junior Cloud Engineer-level infrastructure!